Convert Number To Words

划分测试单元

1. NumberingSystem.max_value

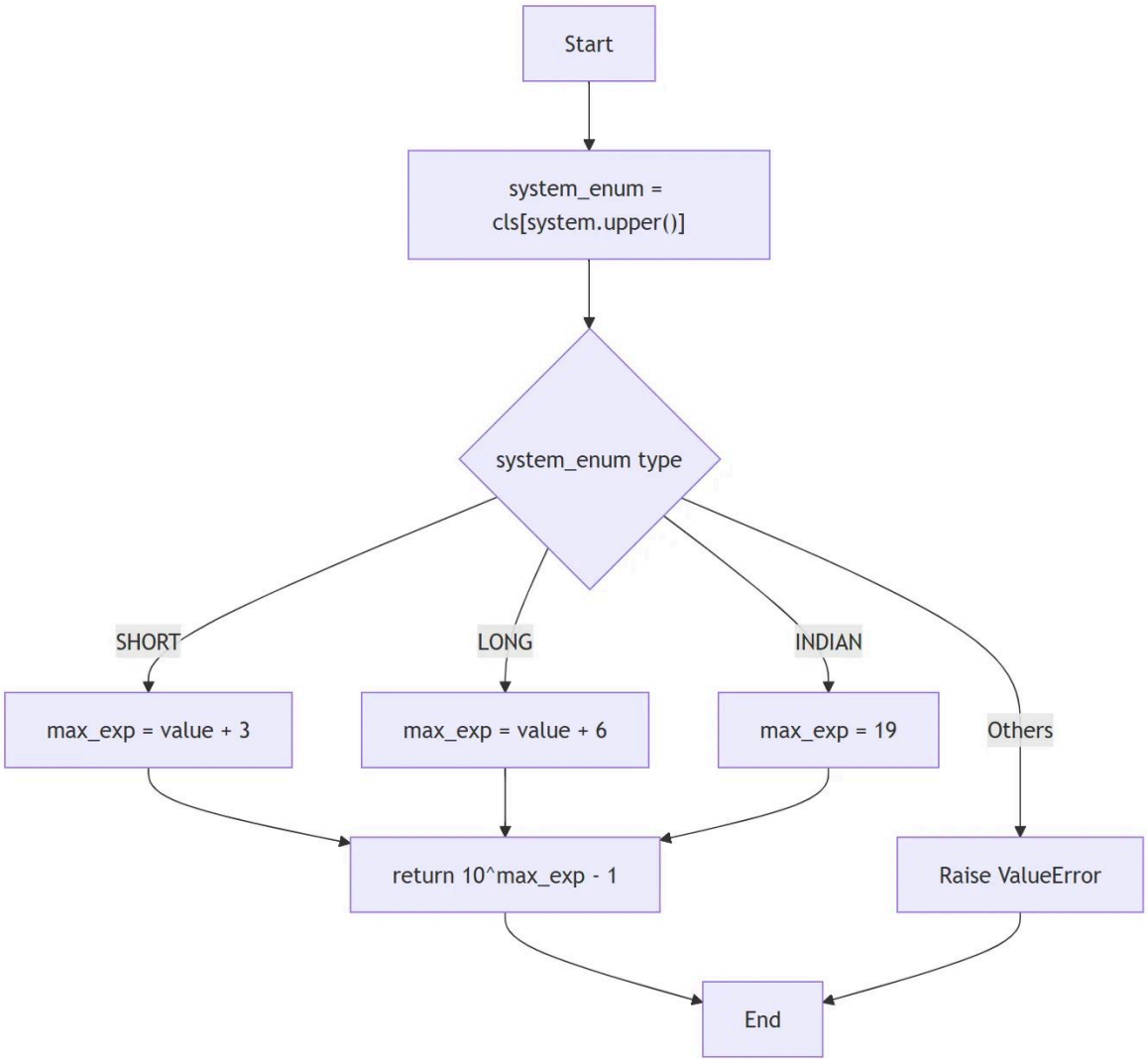
原函数

- 功能：
  - 根据不同的数字命名系统返回该系统支持的最大整数
  - 支持三种系统：
    i. SHORT → 最大值 $10^{18} - 1$
    ii. LONG → 最大值 $10^{21} - 1$
    iii. INDIAN → 最大值 $10^{19} - 1$
- 函数逻辑：
  a. 将输入字符串转为大写并查找对应枚举
  a. 使用 `match-case` 判断系统类型
  b. 根据系统计算最大指数 `max_exp`
  c. 返回最大值 `10**max_exp - 1`
  d. 如果系统不存在 → 抛出异常

Python

```python
# 注释划分基本块
@classmethod
    def max_value(cls, system: str) -> int:
        match system_enum := cls[system.upper()]:  # B1
            case cls.SHORT:  # B2
                max_exp = system_enum.value[0][0] + 3
            case cls.LONG:  # B3
                max_exp = system_enum.value[0][0] + 6
            case cls.INDIAN:  # B4
                max_exp = 19
            case _:  # B5
                raise ValueError("Invalid numbering system")
        return 10**max_exp - 1  # B6
```

控制流图（CFG）



圈复杂度计算

- 相关公式：
  - 圈复杂度公式： $V(G) = E - N + 2$
  - 简化公式： $V(G) = \text{判定节点数} + 1$
- 计算：
  - 判定节点：
    i. `system == SHORT`
    ii. `system == LONG`
    iii. `system == INDIAN`
  - 因此， $V(G) = 3 + 1 = 4$,需要至少 4 条独立路径测试

- Path1：SHORT 系统
  - Start → SHORT → return
- Path2：LONG 系统
  - Start → LONG → return
- Path3：INDIAN 系统
  - Start → INDIAN → return
- Path4：非法系统
  - Start → invalid system → KeyError
    - `cls[system.upper()]` 本身在系统不存在时就会抛出 `KeyError`，而不是走到 `case _` 的 `ValueError`。也就是说 `case _` 实际上是不可达代码，因为非法输入在 match 的赋值阶段就已经崩溃了。

测试用例设计

Test ID	输入	覆盖路径	预期输出
TC1	"short"	Path1	$10^{18} - 1$
TC2	"long"	Path2	$10^{21} - 1$
TC3	"indian"	Path3	$10^{19} - 1$
TC4	"abc"	Path4	KeyError

---

2. convert_small_number

原函数

- 功能：
  - 将 0-99 的非负整数转换为英文数字表示
- 函数逻辑：

  e. 如果输入 `<0` → 抛出异常

  f. 如果输入 `>=100` → 抛出异常

  g. 拆分为 `tens` 和 `ones`

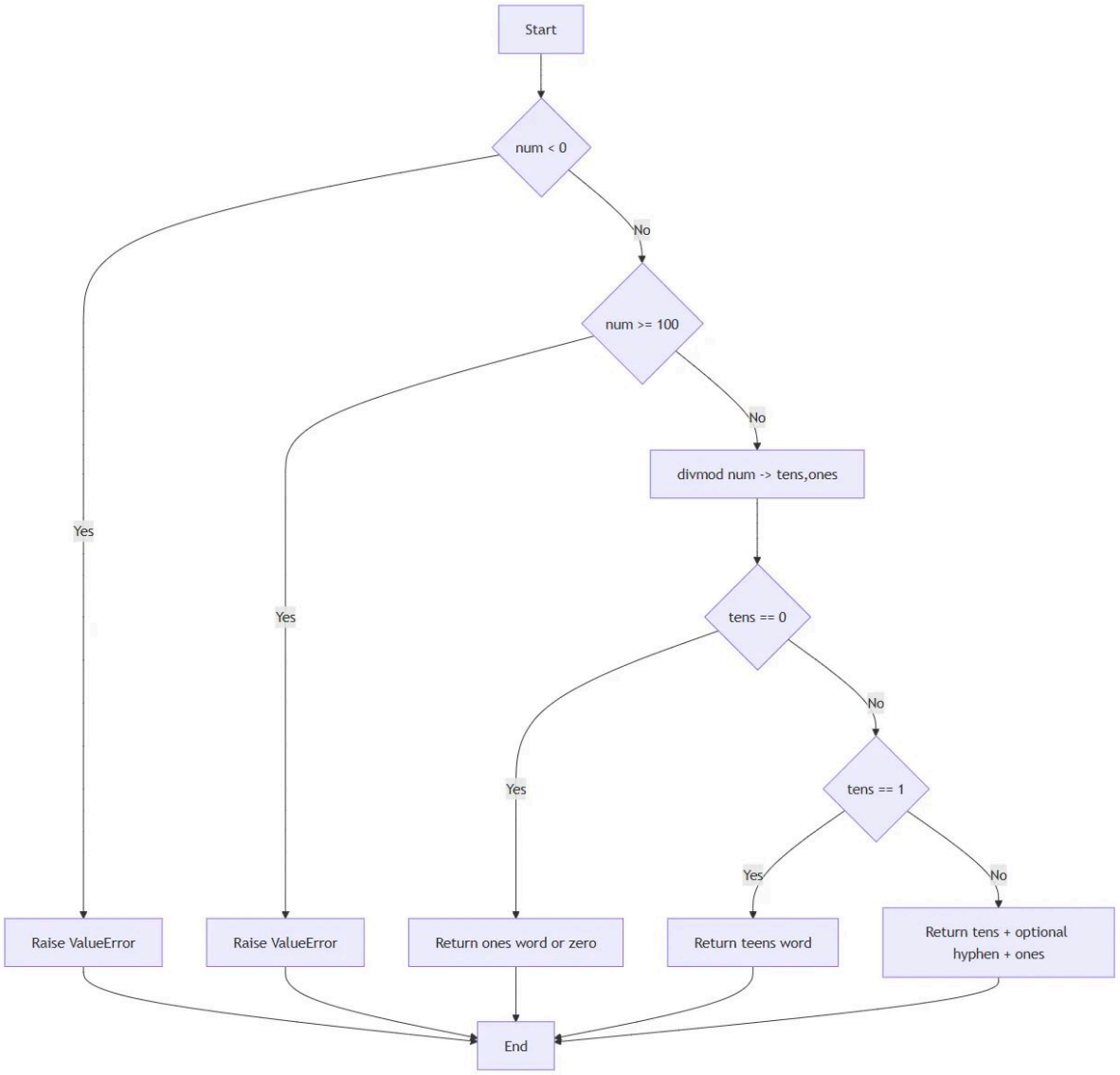
h. 根据不同情况返回不同字符串

- 主要逻辑分支：
  - 输入合法性判断
  - 个位数
  - 10-19
  - 20-99（是否需要连字符）

Python

```python
# 注释划分基本块
def convert_small_number(num: int) -> str:
    if num < 0:  # B1
        raise ValueError("This function only accepts non-negative integers")  # B2
    if num >= 100:  # B3
        raise ValueError("This function only converts numbers less than 100")  # B4
    tens, ones = divmod(num, 10)  # B5
    if tens == 0:  # B6
        return NumberWords.ONES.value[ones] or "zero"  # B7
    if tens == 1:  # B8
        return NumberWords.TEENS.value[ones]  # B9
    return (
        NumberWords.TENS.value[tens]
        + ("-" if NumberWords.ONES.value[ones] else "")
        + NumberWords.ONES.value[ones]
    )  # B10
```

控制流图（CFG）



圈复杂度计算

- 相关公式：
  - 圈复杂度公式： $V(G) = E - N + 2$
  - 简化公式： $V(G) = \text{判定节点数} + 1$
- 计算：
  - 判定节点：
    i. `num < 0`
    ii. `num >= 100`
    iii. `tens == 0`
    iv. `tens == 1`

- 因此， $V(G) = 4 + 1 = 5$,需要至少 5 条独立路径测试
  - Path1：num < 0
    - Start → num<0 → Raise Error
  - Path2：num >= 100
    - Start → num>=100 → Raise Error
  - Path3：个位数
    - Start → num合法 → tens=0 → return ONES
  - Path4：10–19
    - Start → tens=1 → return TEENS
  - Path5：20–99
    - Start → tens>=2 → return tens-word + "-" + ones-word

测试用例设计

> 20 与 42 是同一个 B10 块的不同分支，覆盖有无连字符两种情况

Test ID	输入	覆盖路径	预期输出
TC1	-1	Path1	ValueError
TC2	100	Path2	ValueError
TC3	0	Path3	zero
TC4	7	Path3	seven
TC5	15	Path4	fifteen
TC6	42	Path5	forty-two
TC7	20	Path5	twenty

---

3. convert_number

原函数

- 功能：

- 将任意整数转换为英文数字表达（支持三种数字命名体系）。

- 主要逻辑：

  a. 处理负数

  b. 检查是否超过最大值

  c. 按 power 分组

  d. 每组转换为英文

  e. 拼接结果

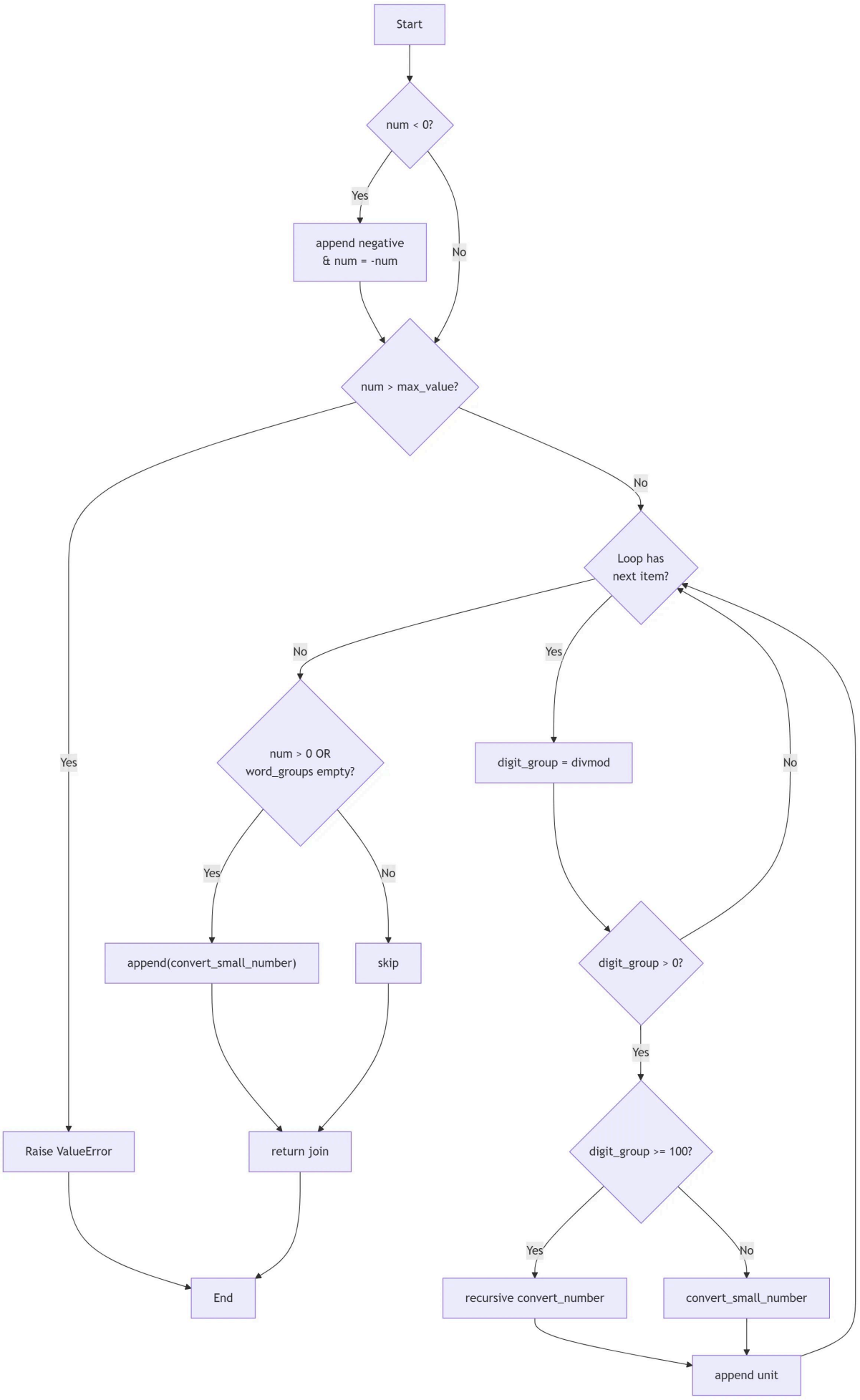
Python

```python
# 注释划分基本块
def convert_number(
    num: int, system: Literal["short", "long", "indian"] = "short"
) -> str:

    word_groups = []

    if num < 0:  # B1
        word_groups.append("negative")  # B2
        num *= -1

    if num > NumberingSystem.max_value(system):  # B3
        raise ValueError("Input number is too large")  # B4

    for power, unit in NumberingSystem[system.upper()].value:  # B5
        digit_group, num = divmod(num, 10**power)  # B6
        if digit_group > 0:  # B7
            word_group = (
                convert_number(digit_group, system)  # B9
                if digit_group >= 100  # B8
                else convert_small_number(digit_group)  # B10
            )
            word_groups.append(f"{word_group} {unit}")  # B11
    if num > 0 or not word_groups:  # B12
        word_groups.append(convert_small_number(num))  # B13
    return " ".join(word_groups)  # B14
```

控制流图（CFG）

> 本函数包含一个`for power, unit in NumberingSystem[system.upper()].value`的遍历循环。在圈复杂度计算中，我们选择不将其单独计入判定节点，原因如下：该循环的迭代次数

> 由`NumberingSystem`枚举的固定结构决定（SHORT系统固定为6组），属于**有界确定性迭代**，其终止条件不依赖运行时输入。测试的关注点在于循环体内的条件分支（`digit_group > 0`、`digit_group >= 100`），而非循环本身的进出。若将循环计入，V(G)=7，需额外增加一条"循环体完全不执行"的路径，但由于枚举结构固定，该路径实际上不可达。因此保守估计V(G)=6，与实际可测路径数一致。


Start
num < 0?
Yes
No
append negative
& num = -num
num > max_value?
Yes
No
Raise ValueError
Loop has
next item?
No
Yes
No
num > 0 OR
word_groups empty?
Yes
No
append(convert_small_number)
skip
return join
End
digit_group = divmod
digit_group > 0?
Yes
No
digit_group >= 100?
Yes
No
recursive convert_number
convert_small_number
append unit

圈复杂度计算

- 相关公式：
  - 圈复杂度公式： $V(G) = E - N + 2$
  - 简化公式： $V(G) = \text{判定节点数} + 1$
- 计算：
  - 判定节点（由于 for 循环属于结构性循环，本实验不单独计入判定节点）：
    i. `num < 0`
    ii. `num > max_value`
    iii. `digit_group > 0`
    iv. `digit_group >= 100`
    v. `num > 0 or word_groups empty`
  - 因此， $V(G) = 5 + 1 = 6$,需要至少 6 条独立路径测试
    - Path1：负数输入
      - Start → num < 0 → 添加negative → 正数转换 → return
    - Path2：超出最大值
      - Start → num > max_value → ValueError
    - Path3：普通小数字
    - Path4：触发递归
      - digit_group>=100 → convert_number 递归
    - Path5：多单位拼接
      - digit_group >0 → convert_small_number → append unit
    - Path6：输入 0
      - word_groups empty → convert_small_number(0) → zero

测试用例设计

Test ID	输入	覆盖路径	预期输出	实际经过的关键分支

TC1	-45	Path1	negative forty-five	
TC2	>max_value	Path2	ValueError	
TC3	42	Path3	forty-two	
TC4	345	Path4	three hundred forty-five	B7(digit_group>0)=T, B8(digit_group>=100)=T, 递归调用convert_number(3)
TC5	12345	Path5	twelve thousand three hundred forty-five	外层：digit_group=12走B8=T递归；内层：digit_group=3走B8=F走B10；B12(num>0)=T继续拼接
TC6	0	Path6	zero	

> TC-12345 是一个**复合路径测试用例**，单次调用同时覆盖了Path4（递归触发）和Path5（多单位拼接）。这是递归函数路径覆盖的固有特性——深层路径的触发必然伴随外层路径的执行。在报告的覆盖率统计中，我们将其计为两条路径均已覆盖，但在与AI方案的对比分析中，这也恰好说明了递归函数的路径设计需要人工仔细推演，AI工具在此类情况下可能产生冗余或遗漏的测试用例。